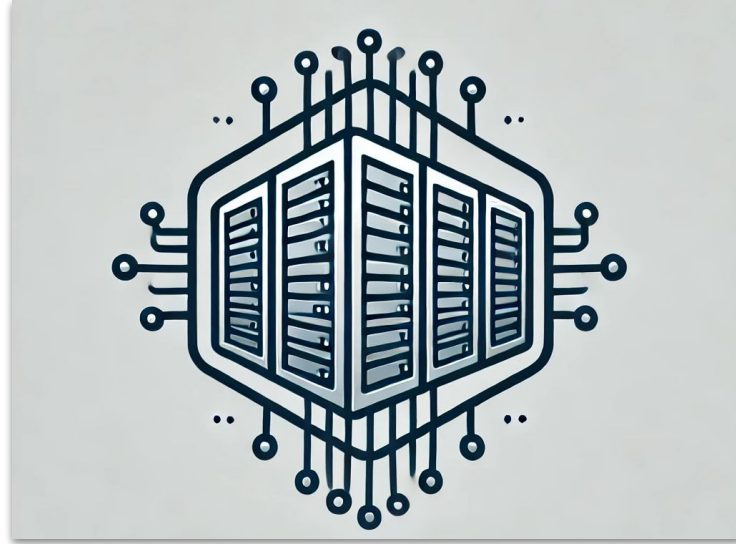


CSC 746: High Performance Computing

Vectorization and Instruction-level Parallelism

17 Sep 2026



Today

Course schedule update

Term project schedule, etc.

Pipelining

Vectorization and Instruction-level Parallelism

CP2 due:
Mon 21 Sep 2026 23:59 PDT

CP2 presentations:
Tue 22 Sep 2026:
Cordano, Bellenberg

Project idea “first light” pitch to
class:
Tue 30 Sep 2025

Course Roadmap/Schedule

Performance Analysis + Optimization	25 Aug – 15 Sep 2026 (3.5 weeks)	2 projects/assignments
SMP on CPUs	17 Sep – 08 Oct 2026 (3.5 weeks)	2 projects/assignments
SMP on GPUs	13 Oct – 29 Oct 2026 (3 weeks)	1 project/assignment
Distributed Memory Parallelism, Advanced Processor Architectures	03 Nov – 19 Nov 2026 (3 weeks)	1 project/assignment
Final Project Presentations	01 Dec – 10 Dec 2026	



Semester Project Schedule

Due	What	Grade %
29 Sep 2026	In-class pre-pre-proposal pitch session	1.0% of semester grade
02 Oct 2026	Project preproposal: problem statement, approach, anticipated results	1.5% of semester grade
12 Oct 2026	Project proposal: problem statement, approach, anticipated results	2.5% of semester grade
01-10 Dec 2026	Project presentations	5% of semester grade
11 Dec 2026	Project deadline: code+report	25% of semester grade

Project Preproposal Idea Pitch Session Tue 29 Sep 2026

We will do this in class on Tue 29 Sep 2026

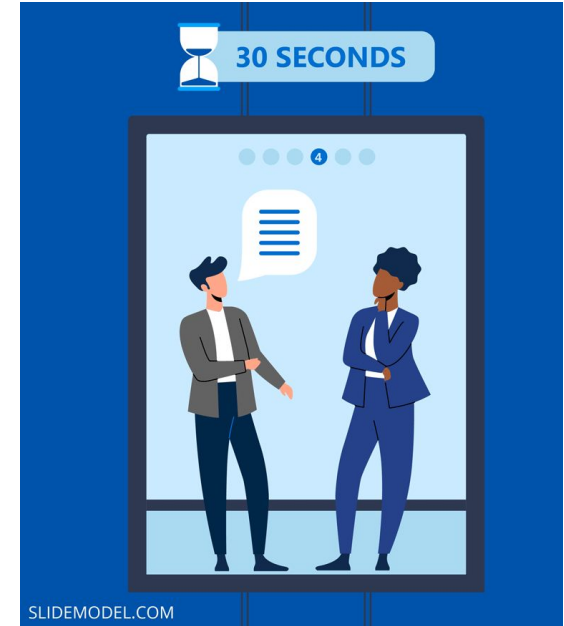
Each person gives a pitch for a project idea or two, you will receive feedback

- No slides
- Just you speaking, ok to use the whiteboard

Desired outcome:

- Everyone gets a better sense of what makes for a good project idea

Turn in: PDF with a problem statement, approach, how you might evaluate your method, potential results



Project Preproposals – Due Friday 02 Oct 2026

1 page of info:

- What problem or question are you studying?
- How will you go about studying the problem?
 - What's the approach?
 - What are you going to implement?
- How will you evaluate the work?
- What are anticipated outcomes?

Can't decide on one? I'll review up to 3 ideas and give you feedback.

Project Ideas

Performance analysis and optimization of a code

- Blocking/tiling, loop reordering
- Metrics: runtime, MFLOP/s, % mem b/w, latency
- Metrics: hardware performance counters: cache miss rate, etc.

Parallelization of a code (shared memory, distributed memory)

- Make it run faster
- Make it accommodate a problem too large for a node (memory)

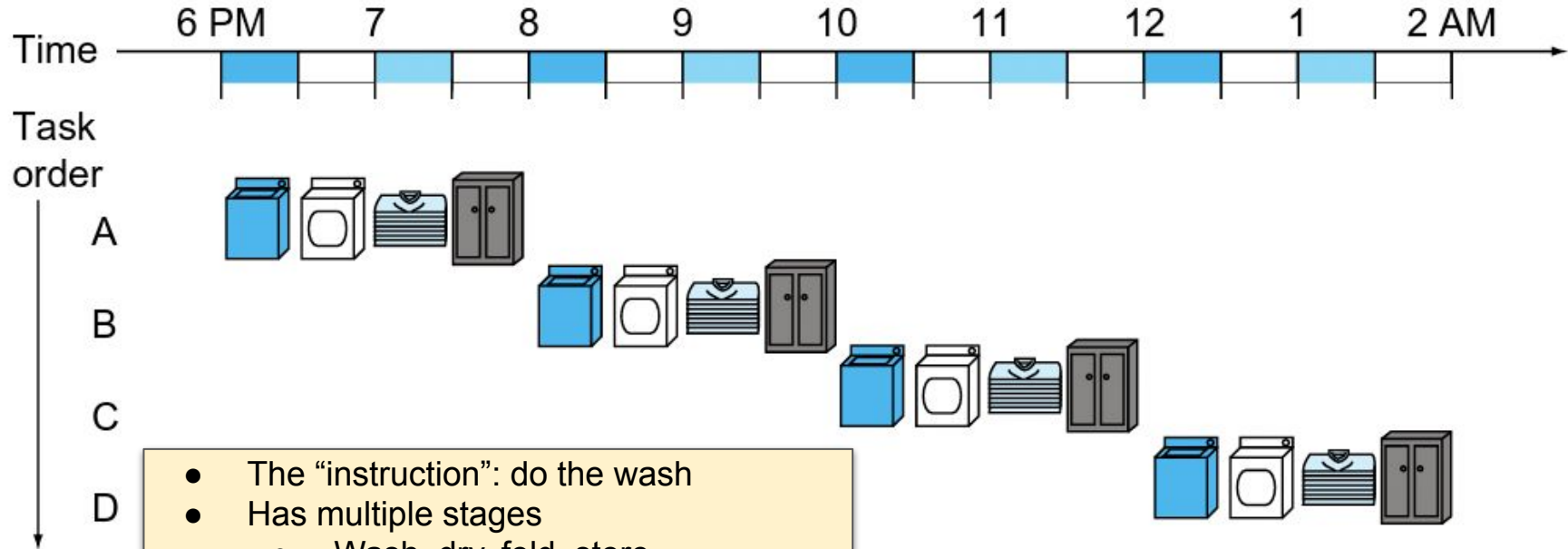
Pipelining

Latency – Measure of Time Required for an Action

What is the *latency* in communication when:

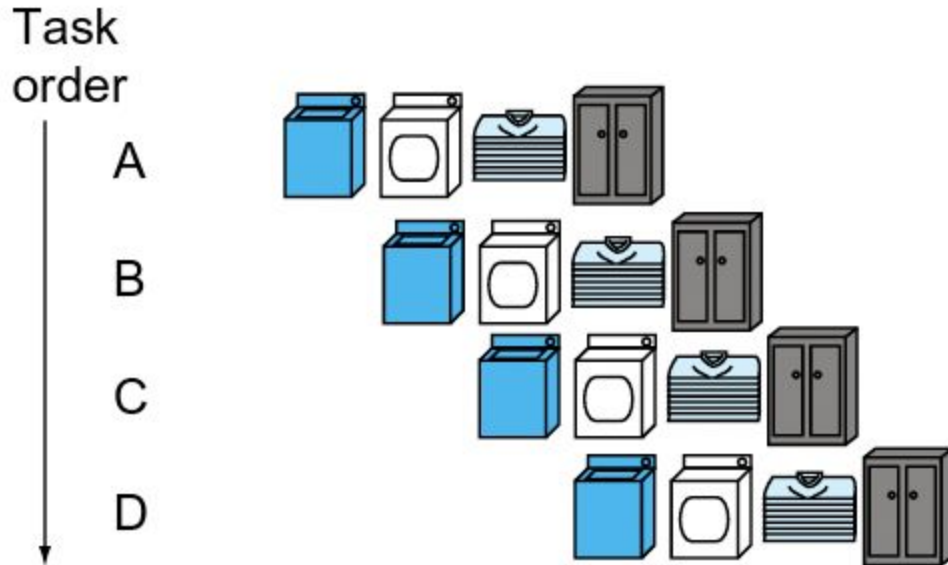
- Sending a letter via US Postal Service?
- Hand carrying a note on a plane from SFO to NYC?
- Speaking a sentence to someone across the room?
- Sending a message via a blinking light using Morse code?

Single Cycle Execution



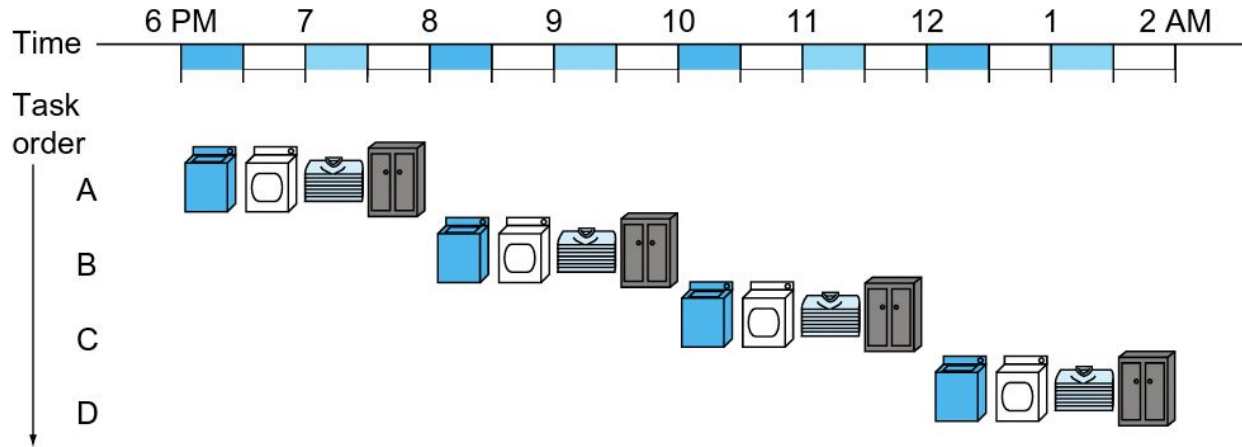
- The “instruction”: do the wash
- Has multiple stages
 - Wash, dry, fold, store
- Each “instruction” must complete before the next “instruction” can begin
- What is the latency of doing a load of laundry in this model?

Pipelined Model



In a pipelined model:

- We still have 4 “instructions”, each of which consists of multiple stages
- As soon as the first resource from Instruction N is released, we can begin Instruction N+1
- Same idea applies to all resources:
 - Wash, dry, fold, store
- What is the latency of doing a load of laundry in this model?

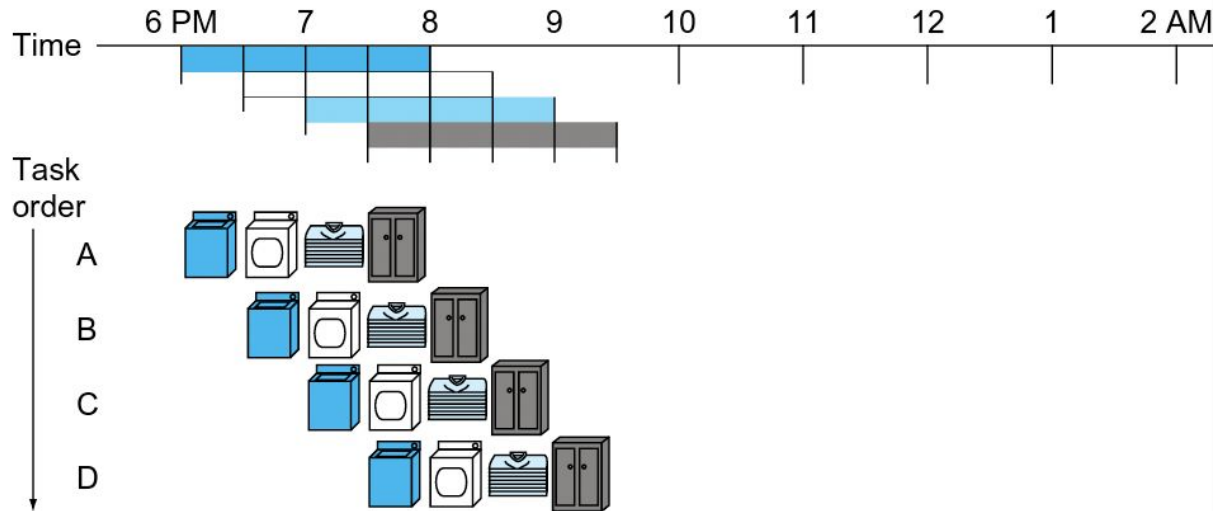
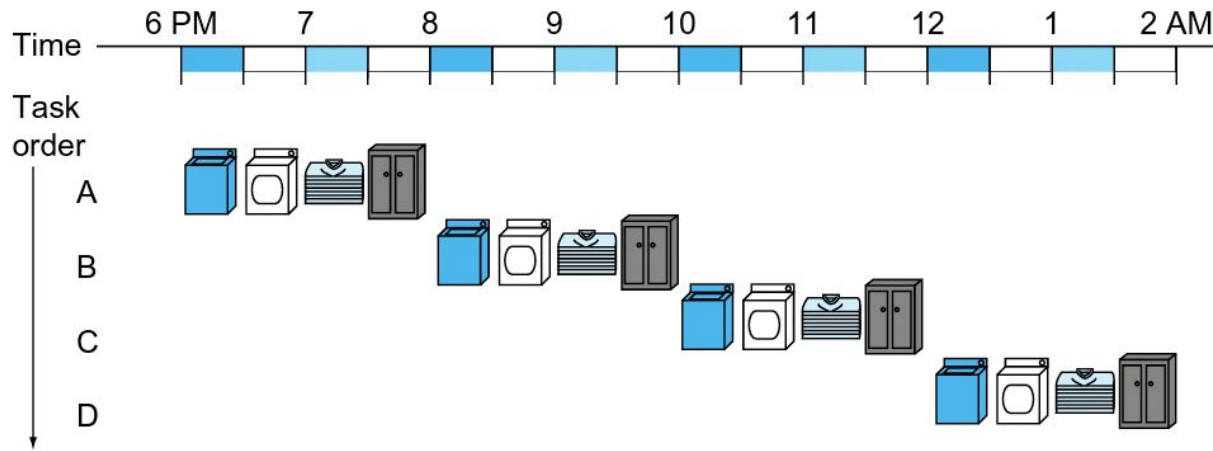


Time required for sequential model:

- Each instruction uses 4 resources, $\frac{1}{2}$ hour each
- Time for 4 instructions:
 - $2 * 4$ instructions
 - = 8 hours

0.5 loads/hour





Time required for sequential model:

- $T = Ntp$
 - $N=4$ instructions
 - t time per stage (0.5hr)
 - $p=4$ pipeline stages
- $T=8$ hrs
- 0.5 loads per hour

Time for pipelined model (4 instr):

- Time for one instruction, plus
- Time for one resource at $N-1$ instructions
- $= 2$ hrs + $1\frac{1}{2}$ hrs
- $= 3\frac{1}{2}$ hrs

2 loads/hour sustained rate

$$T = tp + (N-1)t$$

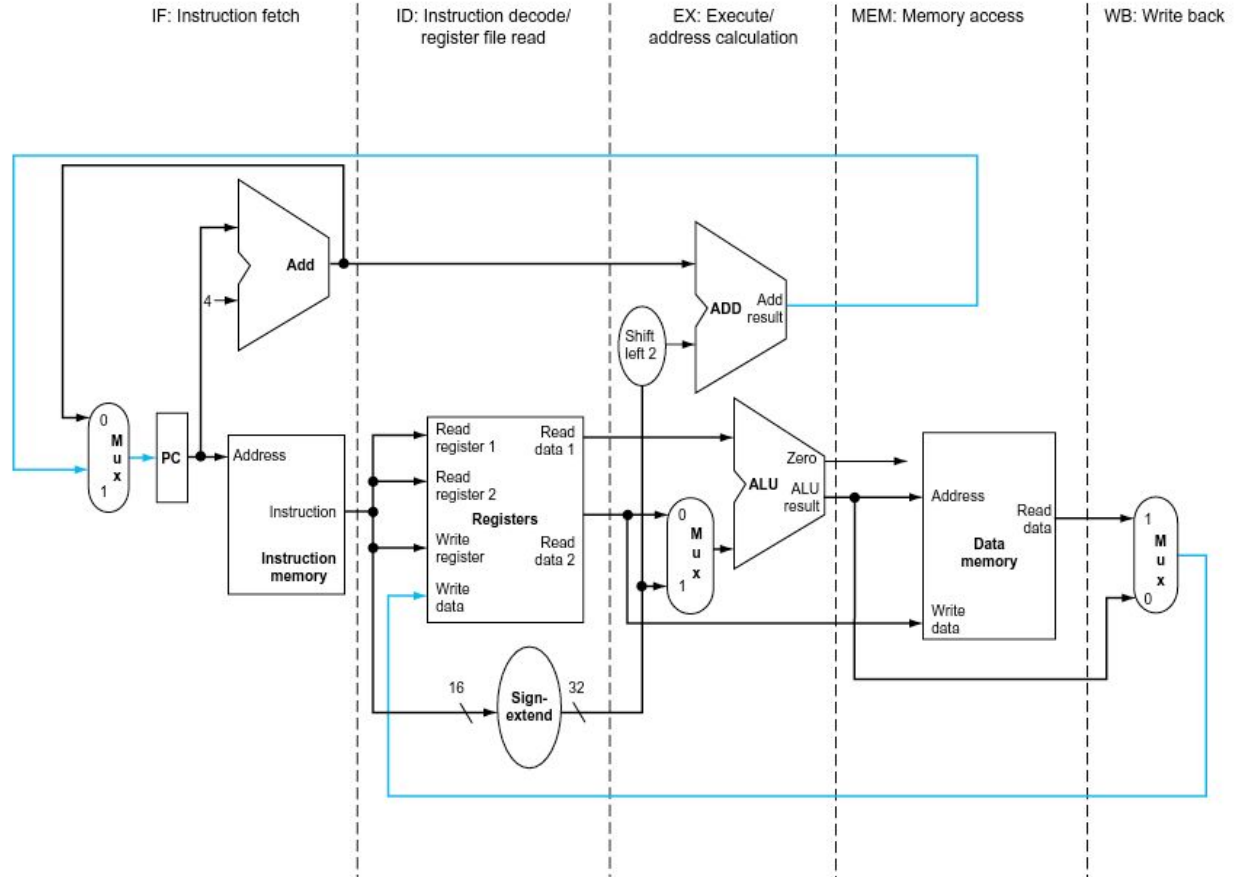
Potential *speedup* =

$$Ntp / (tp + (N-1)t) \approx p \text{ as } N \rightarrow \infty$$

MIPS Pipeline Model

Key idea:

Each of these 5 stages executes in parallel in 1 clock cycle



Simple MIPS Pipeline

IF: instruction fetch from instruction memory

ID: instruction decode/register fetch, generate control signals, get *rs*, *rt*

EX: engage the ALU to compute something

MEM: memory access, read data back from memory, write data to memory

WB: write result back to *rt*

Each stage uses a different part of the datapath

Can overlap steps: pipelined MIPS implementation

Resources May Require Different Amounts of Time

Instruction class	Instruction fetch	Register read	ALU operation	Data access	Register write	Total time
Load word (lw)	200 ps	100 ps	200 ps	200 ps	100 ps	800 ps
Store word (sw)	200 ps	100 ps	200 ps	200 ps		700 ps
R-format (add, sub, AND, OR, slt)	200 ps	100 ps	200 ps		100 ps	600 ps
Branch (beq)	200 ps	100 ps	200 ps			500 ps

All of these instruction types use many of the same resources

Resources May Require Different Amounts of Time

Instruction class	Instruction fetch	Register read	ALU operation	Data access	Register write	Total time
Load word (lw)	200 ps	100 ps	200 ps	200 ps	100 ps	800 ps
Store word (sw)	200 ps	100 ps	200 ps	200 ps		700 ps
R-format (add, sub, AND, OR, slt)	200 ps	100 ps	200 ps		100 ps	600 ps
Branch (beq)	200 ps	100 ps	200 ps			500 ps

Some instructions use slower, or more resources

Resources May Require Different Amounts of Time

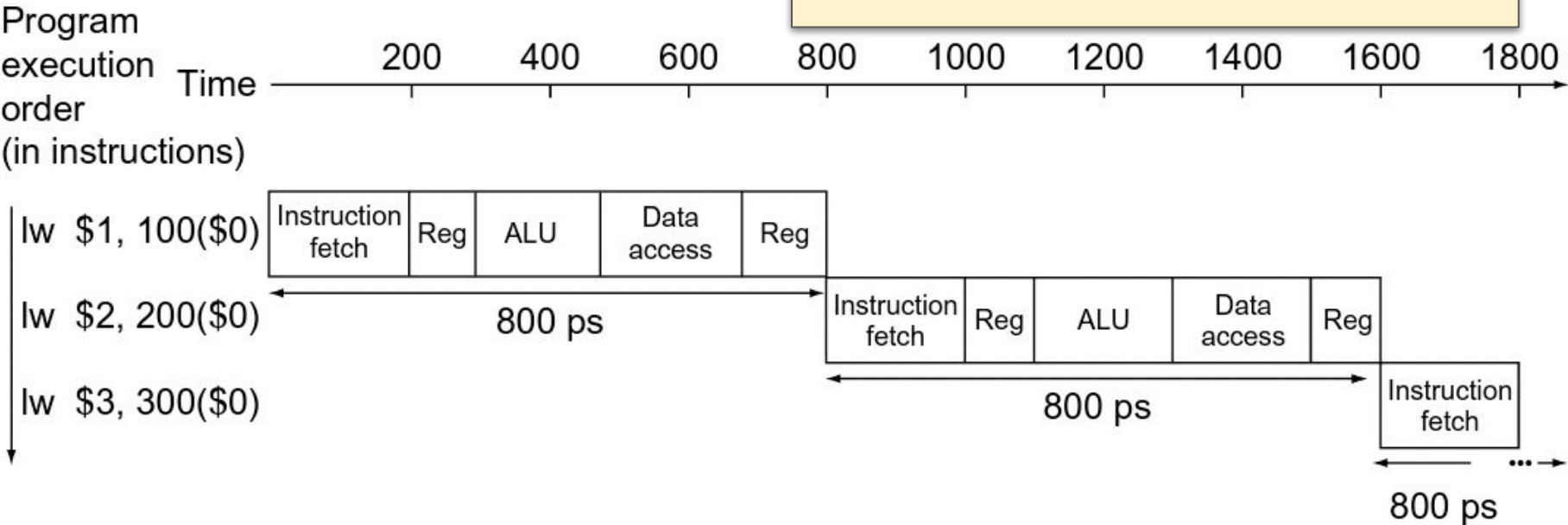
Instruction class	Instruction fetch	Register read	ALU operation	Data access	Register write	Total time
Load word (lw)	200 ps	100 ps	200 ps	200 ps	100 ps	800 ps
Store word (sw)	200 ps	100 ps	200 ps	200 ps		700 ps
R-format (add, sub, AND, OR, slt)	200 ps	100 ps	200 ps		100 ps	600 ps
Branch (beq)	200 ps	100 ps	200 ps			500 ps

Because of differences in resource requirements and their performance, these instructions require varying amounts of time to complete.



Single-cycle, non-pipelined execution

What is the clock cycle time needed for this single-cycle, non-pipelined implementation?

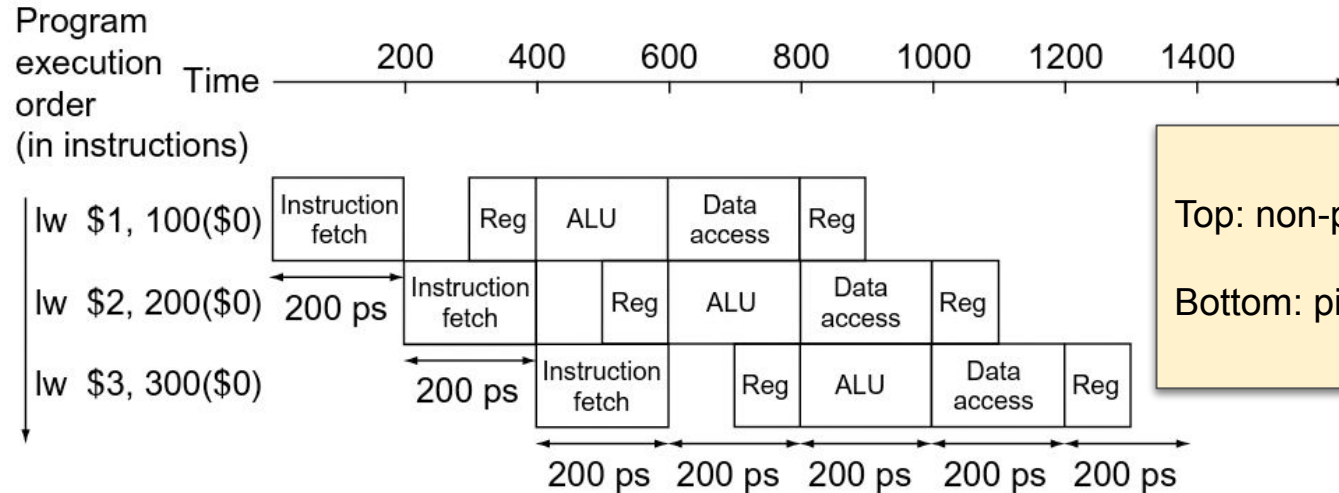
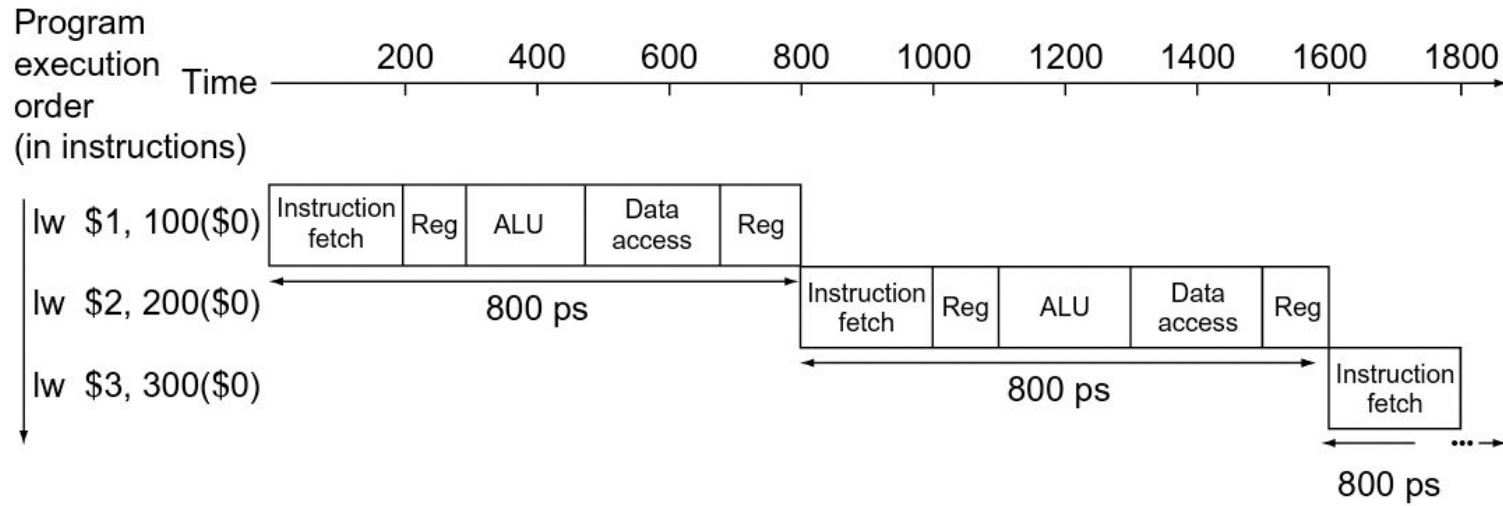


Resources May Require Different Amounts of Time

Instruction class	Instruction fetch	Register read	ALU operation	Data access	Register write	Total time
Load word (lw)	200 ps	100 ps	200 ps	200 ps	100 ps	800 ps
Store word (sw)	200 ps	100 ps	200 ps	200 ps		700 ps
R-format (add, sub, AND, OR, slt)	200 ps	100 ps	200 ps		100 ps	600 ps
Branch (beq)	200 ps	100 ps	200 ps			500 ps

What is the minimum clock cycle time required to accommodate all these instructions?





Top: non-pipelined implementation

Bottom: pipelined implementation

Instruction-level Parallelism

Instruction-level parallelism (ILP)

Pipelining exploits potential parallelism among instructions known as ILP

Two methods for increasing potential amount of ILP:

- Increase pipeline depth to overlap more instructions
 - If wash stage takes most time, then
 - Decompose washing machine into 3 that each perform: wash, rinse, spin
 - Potential speedup: depth of pipeline
- Replicate internal components to launch, process multiple instructions in every pipeline stage
 - “Multiple issue” – scheme where multiple instructions are launched in each clock
 - Washer analogy: replace single washer/dryer with 3 washers/dryers
 - Extra overhead in preparing instructions, data, keeping the components busy

Approaches for Multiple Issue

Issuing multiple instructions per clock

Static multiple issue:

- Decisions made by compiler before execution
- Strategy remains fixed over execution

Dynamic multiple issue:

- Processor makes decisions at runtime

Speculation

Basic idea:

- Want to issue multiple instructions per clock
- Need to identify dependencies between instructions
 - beq results of register comparison impacts subsequent PC
 - Arithmetic operations require first loading something from memory

Using prediction, “guess” about the properties of an instruction

- For instruction reordering
- For provisional resource marshaling

Performed by compiler or by hardware

Static Multiple Issue with MIPS ISA

Instruction pairs: (1) memory load/store, (2) ALU/branch

Use “no op” (nop) instructions when there isn’t a matched pair

Instruction type	Pipe stages							
ALU or branch instruction	IF	ID	EX	MEM	WB			
Load or store instruction	IF	ID	EX	MEM	WB			
ALU or branch instruction		IF	ID	EX	MEM	WB		
Load or store instruction		IF	ID	EX	MEM	WB		
ALU or branch instruction			IF	ID	EX	MEM	WB	
Load or store instruction			IF	ID	EX	MEM	WB	
ALU or branch instruction				IF	ID	EX	MEM	WB
Load or store instruction				IF	ID	EX	MEM	WB

Loop “Unrolling”

A software technique for obtaining more performance from loops that access arrays

Basic idea:

- create multiple copies of the loop body inside the loop itself
- Need to adjust # of iterations, stride of index variable

Helps to set up instructions in a way that make them easier for the compiler and processor to multiple-issue

C++ Example of loop unrolling

Start with vector_sum code:

```
for (i=0;i<N;i++)  
    sum += A[i];
```

```
for (i=0;i<N;i+=4)  
{  
    s1 = A[i];  
    s2 = A[i+1];  
    s3 = A[i+2];  
    s4 = A[i+3];  
    sum += s1+s2+s3+s4;  
}
```



Sample Code (MIPS Assembly Language)

```

Loop: lw    $t0, 0($s1)    # $t0=array element
      addu   $t0,$t0,$s2# add scalar in $s2
      sw     $t0, 0($s1)# store result
      addi   $s1,$s1,-4# decrement pointer
      bne    $s1,$zero,Loop# branch $s1!=0
  
```

	ALU or branch instruction	Data transfer instruction	Clock cycle
Loop:		lw \$t0, 0(\$s1)	1
	addi \$s1,\$s1,-4		2
	addu \$t0,\$t0,\$s2		3
	bne \$s1,\$zero,Loop	sw \$t0, 4(\$s1)	4

Unroll 4x

	ALU or branch instruction	Data transfer instruction	Clock cycle
Loop:		lw \$t0, 0(\$s1)	1
	addi \$s1,\$s1,-4		2
	addu \$t0,\$t0,\$s2		3
	bne \$s1,\$zero,Loop	sw \$t0, 4(\$s1)	4

	ALU or branch instruction	Data transfer instruction	Clock cycle
Loop:	addi \$s1,\$s1,-16	lw \$t0, 0(\$s1)	1
		lw \$t1,12(\$s1)	2
	addu \$t0,\$t0,\$s2	lw \$t2, 8(\$s1)	3
	addu \$t1,\$t1,\$s2	lw \$t3, 4(\$s1)	4
	addu \$t2,\$t2,\$s2	sw \$t0, 16(\$s1)	5
	addu \$t3,\$t3,\$s2	sw \$t1,12(\$s1)	6
		sw \$t2, 8(\$s1)	7
	bne \$s1,\$zero,Loop	sw \$t3, 4(\$s1)	8

Unroll 4x

	ALU or branch instruction	Data transfer instruction	Clock cycle
Loop:		lw \$t0, 0(\$s1)	1
	addi \$s1,\$s1,-4		2
	addu \$t0,\$t0,\$s2		3
	bne \$s1,\$zero,Loop	sw \$t0, 4(\$s1)	4

	ALU or branch instruction	Data transfer instruction	Clock cycle
Loop:	addi \$s1,\$s1,-16	lw \$t0, 0(\$s1)	1
		lw \$t1,12(\$s1)	2
	addu \$t0,\$t0,\$s2	lw \$t2, 8(\$s1)	3
	addu ((\$s1)		4
	addu 5(\$s1)		5
	addu (\$s1)		6
		(\$s1)	7
	bne \$s1,\$zero,Loop	sw \$t3, 4(\$s1)	8

Clock Cycle 1 executes 2 instructions:

- Increment loop index by 4
- Load word from array[0]

Unroll 4x

	ALU or branch instruction	Data transfer instruction	Clock cycle
Loop:		lw \$t0, 0(\$s1)	1
	addi \$s1,\$s1,-4		2
	addu \$t0,\$t0,\$s2		3
	bne \$s1,\$zero,Loop	sw \$t0, 4(\$s1)	4

	ALU or branch instruction	Data transfer instruction	Clock cycle
Loop:	addi \$s1,\$s1,-16	lw \$t0, 0(\$s1)	1
		lw \$t1,12(\$s1)	2
	addu \$t0,\$t0,\$s2	lw \$t2, 8(\$s1)	3
	addu \$t1,\$t1,\$s2	lw \$t3, 4(\$s1)	4
	addu	5(\$s1)	5
	addu	(\$s1)	6
		(\$s1)	7
	bne	(\$s1)	8

Clock Cycle 2 executes 1 instruction:

- Load word for array[3]

Unroll 4x

	ALU or branch instruction	Data transfer instruction	Clock cycle
Loop:		lw \$t0, 0(\$s1)	1
	addi \$s1,\$s1,-4		2
	addu \$t0,\$t0,\$s2		3
	bne \$s1,\$zero,Loop	sw \$t0, 4(\$s1)	4

	ALU or branch instruction	Data transfer instruction	Clock cycle
Loop:	addi \$s1,\$s1,-16	lw \$t0, 0(\$s1)	1
		lw \$t1, 12(\$s1)	2
	addu \$t0,\$t0,\$s2	lw \$t2, 8(\$s1)	3
	addu \$t1,\$t1,\$s2	lw \$t3, 4(\$s1)	4
	addu \$t2,\$t2,\$s2	sw \$t0, 16(\$s1)	5
	addu \$t3,\$t3,\$s2	sw \$t1, 20(\$s1)	6
		sw \$t2, 24(\$s1)	7
	bne \$s1,\$zero,Loop	sw \$t3, 28(\$s1)	8

Clock Cycle 3 executes 2 instructions:

- Add constant \$s2 to array[0]
- Load word array[2]

Unroll 4x

	ALU or branch instruction	Data transfer instruction	Clock cycle
Loop:		lw \$t0, 0(\$s1)	1
			2
			3
			4

Clock Cycle 4 executes 2 instructions:

- Add constant \$s2 to array[3]
- Load word array[1]

	ALU or branch instruction	Data transfer instruction	Clock cycle
Loop:	addi \$s1,\$s1,-16	lw \$t0, 0(\$s1)	1
		lw \$t1,12(\$s1)	2
	addu \$t0,\$t0,\$s2	lw \$t2, 8(\$s1)	3
	addu \$t1,\$t1,\$s2	lw \$t3, 4(\$s1)	4
	addu \$t2,\$t2,\$s2	sw \$t0, 16(\$s1)	5
	addu \$t3,\$t3,\$s2	sw \$t1,12(\$s1)	6
		sw \$t2, 8(\$s1)	7
	bne \$s1,\$zero,Loop	sw \$t3, 4(\$s1)	8

Unroll 4x

	ALU or branch instruction	Data transfer instruction	Clock cycle
Loop:		lw \$t0, 0(\$s1)	1
	addi \$s1,\$s1,-4		2
	addu \$t0,\$t0,\$s2		3
			4

Clock Cycle 5 executes 2 instructions:

- Add constant \$s2 to array[2]
- Store word array[0]
 - Note: bug in this line for sw
 - Should be 0(\$s1) not 16(\$s1)

	ALU or branch instruction		Clock cycle
Loop:	addi \$s1,\$s1,-16	lw \$t0, 0(\$s1)	1
		lw \$t1, 12(\$s1)	2
	addu \$t0,\$t0,\$s2	lw \$t2, 8(\$s1)	3
	addu \$t1,\$t1,\$s2	lw \$t3, 4(\$s1)	4
	addu \$t2,\$t2,\$s2	sw \$t0, 16(\$s1)	5
	addu \$t3,\$t3,\$s2	sw \$t1, 12(\$s1)	6
		sw \$t2, 8(\$s1)	7
	bne \$s1,\$zero,Loop	sw \$t3, 4(\$s1)	8

Unroll 4x

	ALU or branch instruction	Data transfer instruction	Clock cycle
Loop:		lw \$t0, 0(\$s1)	1
	addi \$s1,\$s1,-4		2
	addu \$t0,\$t0,\$s2		3
	bne \$s1,\$zero,Loop	sw \$t0, 4(\$s1)	4

Clock Cycle 6 executes 2 instructions:

- Add constant \$s2 to array[1]
- Store word array[3]

	ALU or branch instruction		Clock cycle
Loop:	addi \$s1,\$s1,-4		1
		lw \$t1,12(\$s1)	2
	addu \$t0,\$t0,\$s2	lw \$t2, 8(\$s1)	3
	addu \$t1,\$t1,\$s2	lw \$t3, 4(\$s1)	4
	addu \$t2,\$t2,\$s2	sw \$t0, 16(\$s1)	5
	addu \$t3,\$t3,\$s2	sw \$t1,12(\$s1)	6
		sw \$t2, 8(\$s1)	7
	bne \$s1,\$zero,Loop	sw \$t3, 4(\$s1)	8

Unroll 4x

	ALU or branch instruction	Data transfer instruction	Clock cycle
Loop:		lw \$t0, 0(\$s1)	1
	addi \$s1,\$s1,-4		2
	addu \$t0,\$t0,\$s2		3
	bne \$s1,\$zero,Loop	sw \$t0, 4(\$s1)	4

Clock Cycle 7 executes 1 instructions:

- Store word array[2]
- (No ALU operation to perform here)

	ALU or branch instruction		Clock cycle
Loop:	addi \$s1,\$s1,-4		1
		lw \$t1,12(\$s1)	2
	addu \$t0,\$t0,\$s2	lw \$t2, 8(\$s1)	3
	addu \$t1,\$t1,\$s2	lw \$t3, 4(\$s1)	4
	addu \$t2,\$t2,\$s2	sw \$t0, 16(\$s1)	5
	addu \$t3,\$t3,\$s2	sw \$t1,12(\$s1)	6
		sw \$t2, 8(\$s1)	7
	bne \$s1,\$zero,Loop	sw \$t3, 4(\$s1)	8

Unroll 4x

	ALU or branch instruction	Data transfer instruction	Clock cycle
Loop:		lw \$t0, 0(\$s1)	1
	addi \$s1,\$s1,-4		2
	addu \$t0,\$t0,\$s2		3
	bne \$s1,\$zero,Loop	sw \$t0, 4(\$s1)	4

Clock Cycle 8 executes 2 instructions:

- Store word array[1]
- BNE instruction

	ALU or branch instruction		Clock cycle
Loop:	addi \$s1,\$s1,-4		1
		lw \$t1,12(\$s1)	2
	addu \$t0,\$t0,\$s2	lw \$t2, 8(\$s1)	3
	addu \$t1,\$t1,\$s2	lw \$t3, 4(\$s1)	4
	addu \$t2,\$t2,\$s2	sw \$t0, 16(\$s1)	5
	addu \$t3,\$t3,\$s2	sw \$t1,12(\$s1)	6
		sw \$t2, 8(\$s1)	7
	bne \$s1,\$zero,Loop	sw \$t3, 4(\$s1)	8

Comparison of Clocks: Rolled vs. Unrolled

	ALU or branch instruction	Data transfer instruction	Clock cycle
Loop:		lw \$t0, 0(\$s1)	1
	addi \$s1,\$s1,-4		2
	addu \$t0,\$t0,\$s2		3
	bne \$s1,\$zero,Loop	sw \$t0, 4(\$s1)	4

4 clocks * N iterations

= 4N clocks

	ALU or branch instruction	Data transfer instruction	Clock cycle
Loop:	addi \$s1,\$s1,-16	lw \$t0, 0(\$s1)	1
		lw \$t1,12(\$s1)	2
	addu \$t0,\$t0,\$s2	lw \$t2, 8(\$s1)	3
	addu \$t1,\$t1,\$s2	lw \$t3, 4(\$s1)	4
	addu \$t2,\$t2,\$s2	sw \$t0, 16(\$s1)	5
	addu \$t3,\$t3,\$s2	sw \$t1,12(\$s1)	6
		sw \$t2, 8(\$s1)	7
	bne \$s1,\$zero,Loop	sw \$t3, 4(\$s1)	8

8 clocks * N/4 iterations

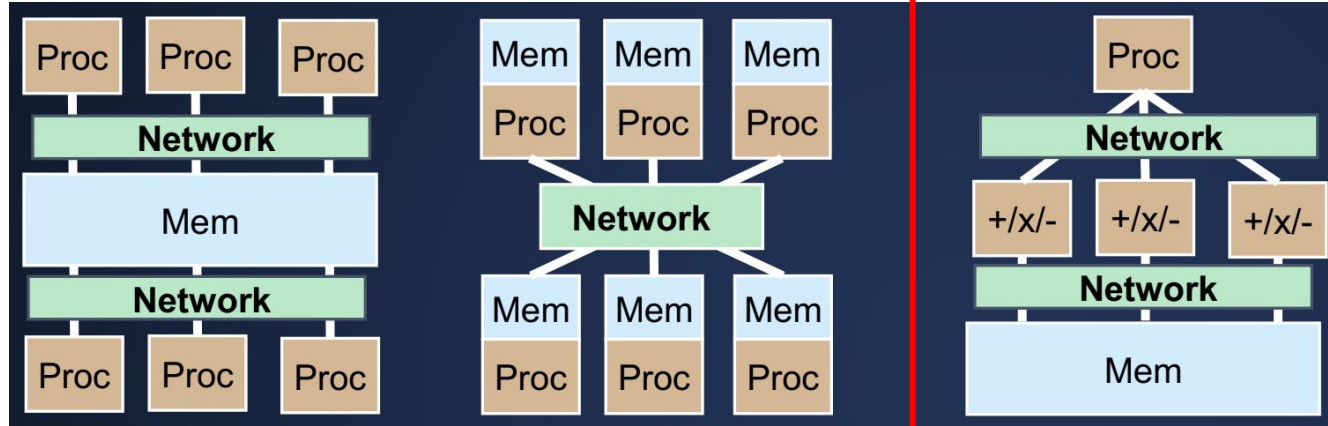
= 2N clocks

This example shows a 2x speedup of code simply by doing loop unrolling



Vectorization

Parallel Machine Models



Abstract
machine
models

Shared memory	Distributed memory	SIMD (Vector)
Processors execute own instruction stream	Processors execute own instruction stream	One instruction stream applied to multiple data
Communicate by reading/writing memory	Communicate by sending messages	Communicate through memory

Vectorization and SIMD Processing

Scalar operations act on individual data elements, one at a time

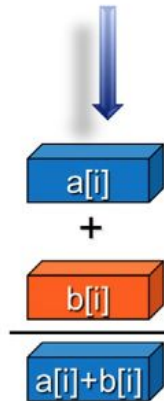
Vector operations are single instructions applied to multiple data items

Single Instruction Multiple Data (SIMD parallelism)

- Caveat: SIMD and vectorization are similar concepts, but are not strictly the same thing

- **Scalar mode**

- One instruction produces one result (SISD)



- **SIMD processing**

- One instruction can produce multiple results (SIMD)
- using AVX VADDPS instruction

```
for (i=0; i<=MAX; i++)  
    c[i] = a[i] + b[i];
```

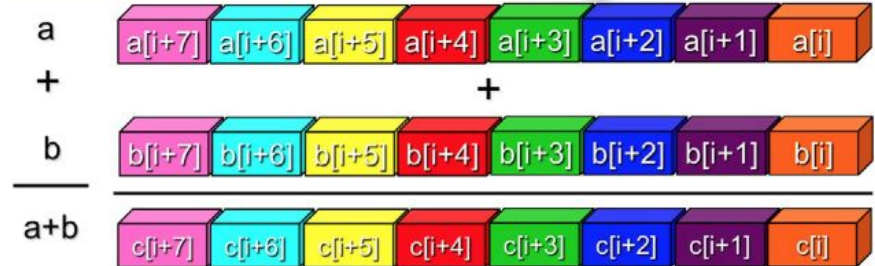


Image credit: Raskulinec & Fiksman, Elsevier 2015
<https://doi.org/10.1016/B978-0-12-803819-2.00006-9>

Vectorization Analogy: Loop Unrolling

Original loop: processes one item at a time

```
for (off_t i=0; i<N; i++)  
    c[i] += a[i]*b[i]
```

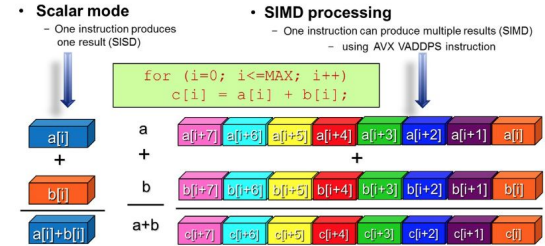
Unrolled loop: processes 4 items at a time

```
for (off_t i=0; i<N; i+=4)  
{  
    c[i] += a[i]*b[i]  
    c[i+1] += a[i+1]*b[i+1]  
    c[i+2] += a[i+2]*b[i+2]  
    c[i+3] += a[i+3]*b[i+3]  
}
```

Vectorization:
performs all these
operations with 1
instruction in “1 tick”

How to Do Vectorization

- (x86) Write assembler that uses AVX-512 instructions
 - `vfmadd231pd zmm1, zmm0, zmm0` // fused multiply add on AVX-512 registers, 16 dp FLOPs
 - Highly platform dependent, not portable
 - Squeeze out last drop of performance
- Use “vector intrinsics”
 - Like assembler, more easily mixed with your C++ code
 - Also not portable
- Automatic compiler vectorization
 - Highly portable, this is what we’ll focus on
 - Can be tricky to get right: you have to write good code
- Use a library that’s already vectorized
 - A good option for portability, rapid code development



Vectorization Coding Strategies

- Focus your attention on sections of code that can actually benefit from vectorization
- Look for loops where you are doing lots of computation, e.g.:

```
for (off_t i=0;i<N;i++)
```

```
    c[i] += a[i]*b[i]
```

- Write code in a form the compiler will actually vectorize
- Use the correct compiler flags to request automatic vectorization
- Verify the compiler vectorized your code
- Implement fixes, hints where the compiler didn't vectorize your code

Basic Requirements for Loops to be Vectorized

- The loop must be countable at runtime
 - N must be known before the loop executes
 - No conditional terminations inside the loop (e.g., break)
- Single control flow within the loop
 - Branching, conditionals, switch statements impede vectorization
 - Workaround: masked operations
- Avoid function calls
 - Exceptions: to functions that can be replaced with inline vector instructions, such as math functions, etc.

Vectorization Data Dependencies: Read-after-write

```
int a[5] = {0,1,2,3,4};  
int b[5] = {5,6,7,8,9};  
  
for (i=1; i<5; i++) {  
    a[i] = a[i-1] + b[i];  
}
```

For this code example, each loop iteration depends on a value completed in the previous loop iteration.

We are “reading after writing”.

If we look at a serial evaluation of this loop (left), we see that it produces the correct results.

Red numbers indicate values written by one iteration, then read by the next.

```
a[1] = a[0] + b[1] → a[1] = 0 + 6 → a[1] = 6  
a[2] = a[1] + b[2] → a[2] = 6 + 7 → a[2] = 13  
a[3] = a[2] + b[3] → a[3] = 13 + 8 → a[3] = 21  
a[4] = a[3] + b[4] → a[4] = 21 + 9 → a[4] = 30  
  
a = {0, 6, 13, 21, 30}
```

```
a[i-1] = {0,1,2,3}    (load)  
b[i]    = {6,7,8,9}    (load)  
{0,1,2,3} + {6,7,8,9} = {6,8,10,12}    (operate)  
a[i] = {6,8,10,12}    (store)
```

a = {0, 6, 8, 10, 12} ≠ {0, 6, 13, 21, 30} **NOT VECTORIZABLE**

When executed in a vectorized fashion, it fails due to the read-after-write dependency because it cannot be computed correctly on vector hardware.

Read-after-write is NOT vectorizable



Vectorization Data Dependencies: Write-after-read

```
int a[5] = {0,1,2,3,4};  
int b[5] = {5,6,7,8,9};  
  
for (i=0; i<4; i++) {  
    a[i] = a[i+1] + b[i];  
}
```

```
a[1] = a[0] + b[1] → a[1] = 1 + 5 → a[1] = 6  
a[2] = a[1] + b[2] → a[2] = 2 + 6 → a[2] = 8  
a[3] = a[2] + b[3] → a[3] = 3 + 7 → a[3] = 10  
a[4] = a[3] + b[4] → a[4] = 4 + 8 → a[4] = 12  
  
a = {6, 8, 10, 12, 4}
```

```
a[i+1] = {1,2,3,4}    (load)  
b[i]    = {5,6,7,8}    (load)  
{1,2,3,4} + {5,6,7,8} = {6,8,10,12}    (operate)  
a[i] = {6,8,10,12}    (store)  
  
a = {6, 8, 10, 12, 4}    VECTORIZABLE
```

The opposite of read-after-write: a value used as input in an earlier iteration, then written in a later iteration.

If we look at a serial evaluation of this loop (left), we see that it produces the correct results.

There are no red numbers as before: none are written in the current iteration then used in a later one.

When executed in a vectorized fashion, we get the same result as if done in serial.

Write-after-read IS vectorizable.



Vectorization Data Dependencies: Write-after-write

```
int a[5] = {0,1,2,3,4};  
int b[5] = {5,6,7,8,9};  
int c[5];  
  
for (i=0; i<4; i++) {  
    c[i%2] = a[i] + b[i];  
}
```

Write-after-write occurs when multiple loop iterations alter the value at a single location.

A value is written by one loop iteration, and then written again by a later iteration.

If an identical address exists for any two or more store operations, the result to be stored is indeterminate.

Write-after-write is NOT vectorizable.



Vectorization Data Dependencies: Read-after-read

```
int a[5] = {0,1,2,3,4};  
int b[5] = {5,6,7,8,9};  
int c[5];  
  
for (i=0; i<4; i++) {  
    c[i] = a[i] + b[i%2];  
}
```

Read-after-read happens when a single vector location is read and used during multiple iterations.

In this case, $b[i\%2]$ may be used during multiple iterations, depending on the value of the loop index variable i .

In principle, there's no problem here because we are not *writing* into b , we are only reading from it; b 's contents don't change over the course of the execution.

Read-after-read IS vectorizable.



Pointer Aliasing: A “Hidden” Data Dependency

```
Void compute(double *a, double *b,  
double *c, int n) {  
    for (i=1;i<n;i++)  
        a[i] = b[i] + c[i];  
}
```

```
/* assume arrays s and t */  
compute(s, s-1, t);
```

```
for (i=1;i<n; i++)  
    s[i] = s[i-1] + t[i];
```

Here we have a function that takes 3 input arrays, *a*, *b*, and *c*.

Is it safe to vectorize the *for* loop?

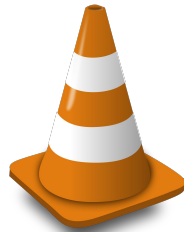
Are there any obvious bad data dependencies?

This is valid syntax, a perfectly valid thing to do.

Inside `compute()`, *a* and *b* refer to overlapping regions of *s*.

Rewriting `compute()` with the variable names *s* and *t* to illustrate the overlap, we quickly see a Read-after-write data dependency.

Concept: avoid use of pointers



Compiler Vectorization Flags on Perlmutter

We are using the g++ compiler (11.2.0) or the Cray g++ wrappers (CC)

Enabling vectorization:

-O2 -ftree-vectorize # enables vectorization, by default produces SSE (128-bit) vector instructions

Reporting:

-fopt-info-vec-all=rpt.txt # generates vectorization report, goes into file rpt.txt

Specifying x86 vector instruction level:

-mavx # use AVX instructions, 128-bit

-mavx2 # use AVX2 instructions, 256-bit

-mavx512f # use AVX512 instructions, 512-bit

Useful info: (1) <https://colfaxresearch.com/knl-avx512/>
(2) <https://gcc.gnu.org/onlinedocs/gcc/x86-Options.html>

Compiler Vectorization Flags on Perlmutter

We are using the g++ compiler (11.2.0) or the Cray g++ wrappers (CC)

Enabling vectorization:

-O2 -ftree-vectorize # enables vectorization, by default produces SSE (128-bit) vector instructions

Reporting:

-fopt-info-vec-all=rpt.txt # generates vectorization

Specifying x86 vector instruction level:

-mavx # use AVX instructions, 128-bit

-mavx2 # use AVX2 instructions, 256-bit

-mavx512f # use AVX512 instructions, 512-bit

Caution:

Don't use these mavx* flags on perlmutter!!

Instead, use -O3 -march=native

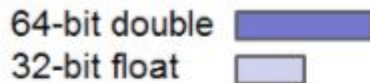
Compiler will generate correct AVX2 instructions

This is the default config when using the RELEASE cmake build type in the code harnesses



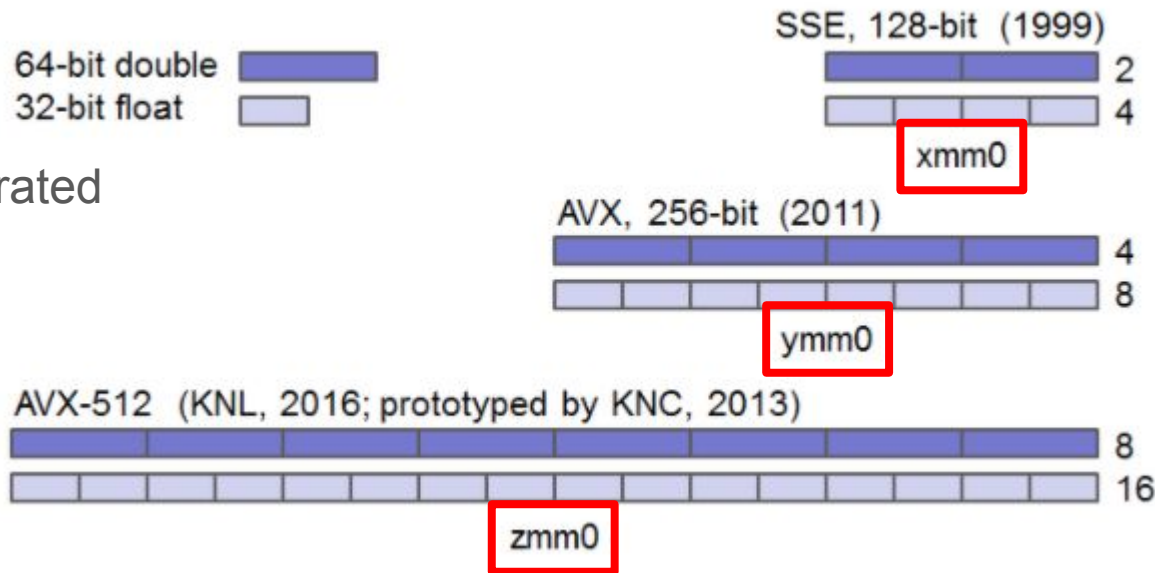
Verify the Compiler Vectorized Your Code

Compiler reports



Look at the assembler generated
by the compiler

- Compiler-generated assembler
- Godbolt.org compiler explorer (live demo)



What are we looking for in the assembly code?
Evidence of scalar, SSE, AVX, or AVX-512
instructions: different sets of registers for each



g++ 11.2.0 -O0 -S → x86 assembly output

```
C++ source #1 ✎ ✕  
A ▾ 📄 + ▾ 🔍 🚫 C++  
1  #include <iostream>  
2  
3  // Type your code here, or load an example.  
4  int sum(int64_t N, int64_t A[])  
5  {  
6      int64_t i, accum=0;  
7  
8      for (i=0;i<N;i++)  
9          accum += A[i];  
10  
11     return accum;  
12 }
```

```
1  sum(long, long*):  
2      push    rbp  
3      mov     rbp, rsp  
4      mov     QWORD PTR [rbp-24], rdi  
5      mov     QWORD PTR [rbp-32], rsi  
6      mov     QWORD PTR [rbp-16], 0  
7      mov     QWORD PTR [rbp-8], 0  
8      jmp     .L2  
9  
10     .L3:  
11     mov     rax, QWORD PTR [rbp-8]  
12     lea     rdx, [0+rax*8]  
13     mov     rax, QWORD PTR [rbp-32]  
14     add     rax, rdx  
15     mov     rax, QWORD PTR [rax]  
16     add     QWORD PTR [rbp-16], rax  
17     add     QWORD PTR [rbp-8], 1  
18  
19     .L2:  
20     mov     rax, QWORD PTR [rbp-8]  
21     cmp     rax, QWORD PTR [rbp-24]  
22     jl      .L3  
23     mov     rax, QWORD PTR [rbp-16]  
24     pop     rbp  
25     ret
```



g++

How many memory accesses?

→ 7 memory accesses per loop !!

How many bytes/access?

→ accesses are 8 bytes (mov qword ptr)

Memory access and compute pattern:

→ one item at a time

C++ source

A ▾

```
1
2
3 // Type your code here, or load an example.
4 int sum(int64_t N, int64_t A[])
5 {
6     int64_t i, accum=0;
7
8     for (i=0; i<N; i++)
9         accum += A[i];
10
11     return accum;
12 }
```

put

sum(long, long*):

```
push    rbp
mov     rbp, rsp
mov     QWORD PTR [rbp-24], rdi
mov     QWORD PTR [rbp-32], rsi
mov     QWORD PTR [rbp-16], 0
mov     QWORD PTR [rbp-8], 0
jmp     .L2
```

.L3:

```
mov     rax, QWORD PTR [rbp-8]
lea     rdx, [0+rax*8]
mov     rax, QWORD PTR [rbp-32]
add     rax, rdx
mov     rax, QWORD PTR [rax]
add     QWORD PTR [rbp-16], rax
add     QWORD PTR [rbp-8], 1
```

.L2:

```
mov     rax, QWORD PTR [rbp-8]
cmp     rax, QWORD PTR [rbp-24]
jl      .L3
mov     rax, QWORD PTR [rbp-16]
pop     rbp
ret
```



g++ 11.2.0 -O2 -free-vectorize -S → x86 assembly output

How many memory accesses?
→ 1 memory accesses per loop

How many bytes/access?
→ accesses are 16 bytes (movdqu)

Memory access and compute pattern:
→ two items at a time (paddq)

```
1  sum(long, long*):
2      test    rdi, rdi
3      jle     .L6
4      cmp     rdi, 1
5      je      .L7
6      mov     rdx, rdi
7      mov     rax, rsi
8      pxor    xmm0, xmm0
9      shr     rdx
10     sal     rdx, 4
11     add     rdx, rsi
12     .L4:
13         movdqu xmm2, XMMWORD PTR [rax]
14         add     rax, 16
15         paddq   xmm0, xmm2
16         cmp     rdx, rax
17         jne     .L4
18         movdqa  xmm1, xmm0
19         mov     rdx, rdi
20         psrldq  xmm1, 8
21         and     rdx, -2
22         and     edi, 1
23         paddq   xmm0, xmm1
24         movq    rax, xmm0
25         je      .L10
26     .L3:
27         add     rax, QWORD PTR [rsi+rdx*8]
28         ret
29     .L10:
30         ret
```



g++ 11.2.0 -O2 -free-vectorize -mavx2 -S → x86 assembly output

How many memory accesses?

→ 1 memory accesses per loop

How many bytes/access?

→ accesses are 32 bytes (YMMWORD PTR)

Memory access and compute pattern:

→ 4 items at a time (vpaddq ymm1)

```
1  sum(long, long*):
2      test    rdi, rdi
3      jle     .L7
4      lea     rax, [rdi-1]
5      cmp     rax, 2
6      jbe     .L8
7      mov     rdx, rdi
8      mov     rax, rsi
9      vpxor   xmm1, xmm1, xmm1
10     shr     rdx, 2
11     sal     rdx, 5
12     add     rdx, rsi
13     .L4:
14     vpaddq   ymm1, ymm1, YMMWORD PTR [rax]
15     add     rax, 32
16     cmp     rax, rdx
17     jne     .L4
18     vmovdqa xmm0, xmm1
19     vextracti128    xmm1, ymm1, 0x1
20     mov     rdx, rdi
21     vpaddq   xmm0, xmm0, xmm1
22     and     rdx, -4
23     vpsrlq   xmm1, xmm0, 8
```



g++ 11.2.0 -O2 -free-vectorize -mavx512f -S

→ **x86 assembly output**

How many memory accesses?

→ 1 memory accesses per loop

How many bytes/access?

→ accesses are 64 bytes (ZMMWORD PTR)

Memory access and compute pattern:

→ 8 items at a time (vpaddq zmm0)

```
1  sum(long, long*):
2      test    rdi, rdi
3      jle     .L7
4      lea     rax, [rdi-1]
5      cmp     rax, 6
6      jbe     .L8
7      mov     rdx, rdi
8      mov     rax, rsi
9      vpxor   xmm0, xmm0, xmm0
10     shr     rdx, 3
11     sal     rdx, 6
12     add     rdx, rsi
13 .L4:
14     vpaddq   zmm0, zmm0, ZMMWORD PTR [rax]
15     add     rax, 64
16     cmp     rax, rdx
17     jnc     .L4
```



```
g++ -O2 -ftree-vectorize -fopt-info-vec-all -c svcs.cpp
```

Vectorization Report Output Shows Use of SSE Instructions

```
10
11 int64_t
12 sum(int64_t N, uint64_t A[])
13 {
14     // printf(" inside sum_vector perform_sum, N=%lld \n", N);
15     int64_t i, accum=0;
16
17     for (i=0; i<N; i++)
18         accum += A[i];
19
20     return accum;
21 }
```

```
wes@perlmutter:login08:~/SFSU/sum_harness_answerkey> g++ -O2 -ftree-vectorize -fopt-info-vec-all -c svcs.cpp
svcs.cpp:17:14: optimized: loop vectorized using 16 byte vectors
svcs.cpp:12:1: note: vectorized 1 loops in function.
svcs.cpp:20:11: note: ***** Analysis failed with vector mode V2DI
svcs.cpp:20:11: note: ***** Skipping vector mode V16QI, which would repeat the analysis for V2DI
/opt/cray/pe/gcc/11.2.0/snos/include/g++/iostream:74:25: missed: statement clobbers memory: std::ios_base::Init::Init (&__ioinit);
/opt/cray/pe/gcc/11.2.0/snos/include/g++/iostream:74:25: missed: statement clobbers memory: __cxxabiv1::__cxa_atexit (__dt_comp , &__ioinit, &__dso_handle);
svcs.cpp:21:1: note: ***** Analysis failed with vector mode VOID
```

```
g++ -O2 -march=native -ftree-vectorize -fopt-info-vec-all -c svs.cpp
```

Vectorization Report Output Shows Use of AVX/AVX2 Instructions

```
10
11 int64_t
12 sum(int64_t N, uint64_t A[])
13 {
14     // printf(" inside sum_vector perform_sum, N=%lld \n", N);
15     int64_t i, accum=0;
16
17     for (i=0;i<N;i++)
18         accum += A[i];
19
20     return accum;
21 }
```

```
wes@perlmutter:login08:~/SFSU/sum_harness_answerkey> g++ -O2 -march=native -ftree-vectorize -fopt-info-vec-all -c sv
s.cpp
```

```
svs.cpp:17:14: optimized: loop vectorized using 32 byte vectors
```

```
svs.cpp:12:1: note: vectorized 1 loops in function.
```

```
svs.cpp:20:11: note: ***** Analysis failed with vector mode V4DI
```

```
svs.cpp:20:11: note: ***** Skipping vector mode V32QI, which would repeat the analysis for V4DI
```

```
/opt/cray/pe/gcc/11.2.0/snos/include/g++/iostream:74:25: missed: statement clobbers memory: std::ios_base::Init::Ini
t (&__ioinit);
```

```
/opt/cray/pe/gcc/11.2.0/snos/include/g++/iostream:74:25: missed: statement clobbers memory: __cxxabiv1::__cxa_atexit
(__dt_comp , &__ioinit, &__dso_handle);
```

```
svs.cpp:21:1: note: ***** Analysis failed with vector mode VOID
```

Guidelines for Writing Vectorizable Code

Do:

- Use simple for loops
- Straight-line code in loops (avoid conditionals, breaks, returns)
- Use vector-based data structures (arrays)
- Use array notation rather than pointers
- Use loop index variable directly as a subscript
- Access memory efficiently: unit stride, avoid indirect addressing, align data to 16-byte boundaries:
 - `__declspec(align(16)) double a[N];`
- Choose data layout with care

Avoid:

- Function calls (other than math library calls)
- read-after-write dependencies
- Pointers
- Non-vectorizable operations
- Mixing vectorizable types in the same loop
- Data-dependent operations, including loop termination

Vectorization Thoughts

Vectorization can produce significant performance gains

Auto-vectorizing compilers are the way to go: portability

Requires that you pay close attention to how you write your code

You will be studying the performance impact of vectorization in CP#3

More Information on Vectorization

Computer Architecture, Hennessey and Patterson, 5th ed., 2012

Section 4.2: Vector Architecture

https://csu-sfsu.primo.exlibrisgroup.com/permalink/01CALS_SFR/7tr8po/cdi_safari_books_v2_9780123838735

AVX Lineage

Source: <https://en.wikipedia.org/wiki/AVX-512>

Name	Extension sets	Registers	Types
Legacy SSE	SSE–SSE4.2	xmm0–xmm15	single floats <i>from SSE2</i> : bytes, words, doublewords, quadwords and double floats
AVX-128 (VEX)	AVX, AVX2	xmm0–xmm15	bytes, words, doublewords, quadwords, single floats and double floats
AVX-256 (VEX)	AVX, AVX2	ymm0–ymm15	single float and double float <i>from AVX2</i> : bytes, words, doublewords, quadwords
AVX-128 (EVEX)	AVX-512VL	xmm0–xmm31 (k0–k7)	doublewords, quadwords, single float and double float <i>with AVX512BW</i> : bytes and words. <i>with AVX512-FP16</i> : half float
AVX-256 (EVEX)	AVX-512VL	ymm0–ymm31 (k0–k7)	doublewords, quadwords, single float and double float <i>with AVX512BW</i> : bytes and words. <i>with AVX512-FP16</i> : half float
AVX-512 (EVEX)	AVX-512F	zmm0–zmm31 (k0–k7)	doublewords, quadwords, single float and double float <i>with AVX512BW</i> : bytes and words <i>with AVX512-FP16</i> : half float



The End



A Warner Bros.
PICTURE

